자료구조

02 주차

파이썬 리뷰



2장. 파이썬 리뷰



2.1 파이썬이란?

2.2 자료형, 리터럴과 변수

2.3 파이썬의 연산

2.4 함수 호출과 입출력 함수

2.5 제어 구조와 반복

2.6 컬렉션 자료형

2.7 사용자 정의 함수

2.8 변수의 범위

2.9 모듈과 이름 공간

2.10 클래스

2.11 연산자 중복

2.12 상속

파이썬 정리

2.1 파이썬이란?

- AI와 빅데이터의 부상과 함께 최근 각광받고 있는 언어

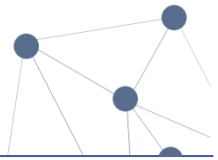
TIOBE

Schedule a demo

Jan 2023	Jan 2022	Change	Programming Language	Ratings	Change
1	1		Python	16.36%	+2.78%
2	2		C	16.26%	+3.82%
3	4	▲	C++	12.91%	+4.62%
4	3	▼	Java	12.21%	+1.55%
5	**5**		**C#**	**5.73%**	**+0.05%**

  - 1991년에 귀도 반 로섬(Guido van Rossum)
  - 문법이 쉬움. 코드를 보면 직관적으로 알 수 있는 부분이 많음
  - 인터프리터 방식 ←→ 컴파일 방식
  - 스크립트 모드 지원
  - 통합개발환경: IDLE, 주피터 노트북, 파이참, 비주얼 스튜디오

2.2 자료형, 리터럴과 변수



- 파이썬 예약어(keyword)

```
and       as      assert    break     class     continue    def
del       elif    else      except    finally   for         from
global    if      import    in        is        lambda      nonlocal
not       or      pass      raise     return    try         while
with      yield   False     None      True
```

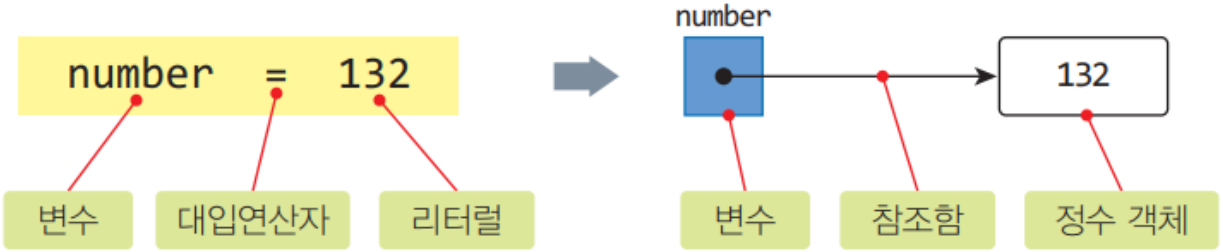
- 리터럴과 자료형

분류	내장 자료형	리터럴의 예
수치	정수(int)	10 -30 0xfffe 073
	실수(float)	3.14 -0.45 123.032E-13
	복소수(complex)	complex(1,2) 1+2j 4+5j
	부울(bool)	True False
시퀀스	문자열(str)	'game' "over" "C"
	리스트(list)	[] [0, 1, 2, 3] [0, 'hello', 3.14]
	튜플(tuple)	(0, 1, 2, 3) ('hello', 'world', 'game')
매핑	딕셔너리(dict)	{ 3.14 : "phi", 4.5 : "score" }
집합	집합(set, frozenset)	{ 1, 2, 3 } {'one', 'two', 'three' }

- 변수(variable)

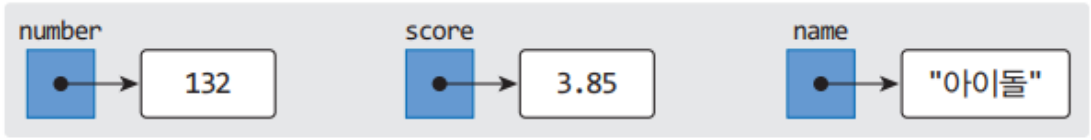
```
number = 132                    # 변수 생성 및 사용 문장
number = number + 8             # 변수 사용 문장
pi = 3.14                       # float 변수
comp = 1 + 2j                   # complex 변수
isValid = True                  # bool 변수
msg = 'game over !!!'           # str 변수
A = [0, 1, 1, 2, 3, 5, 8, 13 ]  # list 변수
```

- 파이썬에서 변수란?
  - 파이썬에서는 모든 자료가 클래스로부터 만들어진 객체
  - 변수는 다른 객체를 참조하는 참조자 또는 포인터의 역할

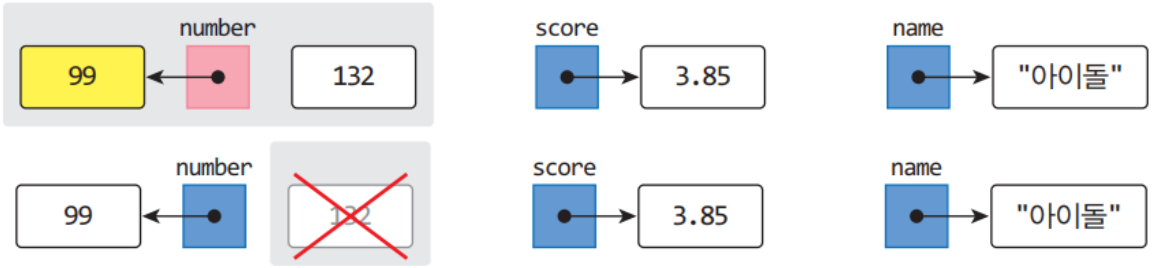


- 변수의 동작 정확히 이해하기

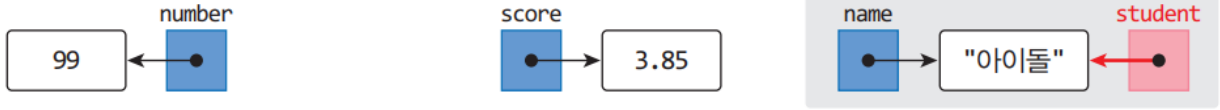
```
number = 132
score = 3.85
name = "아이돌"
```



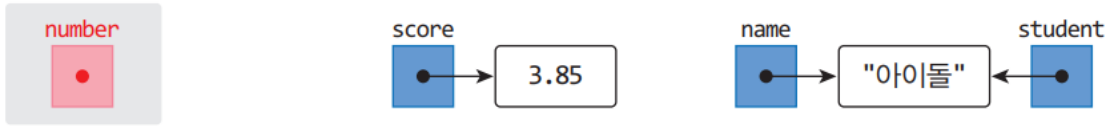
number = 99



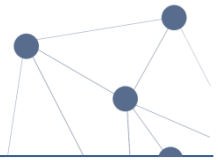
student = name



number = None



2.3 파이썬의 연산

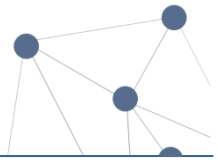


- 나눗셈 연산자 변경
  - 연산자 / : 실수의 나눗셈 (결과가 실수)
  - 연산자 //: 정수 연산(floor division)
- 이항 연산자 ** 추가
- 단항 연산자 ++, -- 제공 않음
  - x++가 아니라 x += 1
- 관계 연산자 >, <, >=, <=, ==, !=
- 불리언 연산자: ||, &&, ! → or, and, not
- in과 not in 연산자

```
'a' in 'banana'          # True
'seed' in 'banana'       # False
```

```
A = [0, 1, 1, 2, 3, 5, 8, 13 ]
if 3 in A :
while 4 in A :
```

2.4 함수 호출과 입출력 함수



- 함수 호출

```
y = sum ( a, b )                # 함수 sum 호출. 인수 2개
w = x * z + func( a, b, c )     # 함수 func 호출. 인수 3개
print( "game over" )            # 화면출력함수 print 호출.
```

- 키보드 입력 함수: input()

```
name = input("당신의 이름을 입력하세요.")          # 문자열
hobby = input("취미가 무엇입니까?")                # 문자열

age = int( input("나이가 몇 살입니까?"))                # 입력 문자열을 정수로 변환
score = float( input("평균 학점이 얼마입니까?"))        # 입력 문자열을 실수로 변환
```

- 화면 출력 함수: print()

```
print( "game over" )
```

```
print("game", end=" ")
print("over")
```

\\ Backslash (\)	\n Newline	\" Double quote (")
\ Single quote (')	\t Horizontal tab	

2.5 제어 구조와 반복



- 분기(branching): if, else, elif

```
if value % 2 == 0 :
    print("짝수입니다.")
```

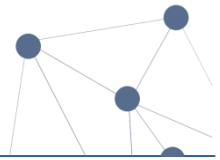
```
if score >= 90 :
    print("합격입니다.")
    print("장학금 대상입니다.")
```

```
if score >= 90 :
    print("합격입니다.")
print("장학금 대상입니다.")
```

```
if value % 2 == 0 : print("짝수입니다.")        # 블록을 한 줄로 처리
else : print("홀수입니다.")                    # 블록을 한 줄로 처리
```

```
if score >= 90 :
    grade = "A"
else :
    if score >= 80 :
        grade = "B"
    else :
        if score >= 70 :
            grade = "C"
        else :
            if score >= 60 :
                grade = "D"
            else:
                grade = "F"
```

```
if score >= 90 :
    grade = "A"
elif score >= 80 :
    grade = "B"
elif score >= 70 :
    grade = "C"
elif score >= 60 :
    grade = "D"
else :
    grade = "F"
```



- 반복(looping)

```
dan = int(input("구구단 단 입력: ")
for n in range( 2, 10, 1 ) :
    print( "%2d x %2d = "% (dan, n), dan*n )
```

```
dan = int(input("구구단 단 입력: ")
n = 2
while n < 10 :
    print( "%2d x %2d = "% (dan, n), dan*n )
    n += 1
```

- Range()

```
for n in range( 5 ) :          # n: 0, 1, 2, 3, 4
for n in range( 2, 10 ) :      # n: 2, 3, ..., 9
for n in range( 10, 3, -2 ) :  # n: 10, 8, 6, 4
```

```
for item in [12, 33, 52, 26, 99] :   # 리스트의 모든 항목에 대해 반복
    print( "값 =", item)             # 12, 33, 52, 26, 99 출력
```

```
for c in "Game Over !" :   # 문자열의 각 문자에 대해
    print( "값 =", c)
```

```
mySet = set([12, 33, 52, 26, 99])
for e in mySet :
    print( "값 =", e)
```

```
myDict = { 'A':12, 'B':33, 'C':52, 'D':26, 'E':99 }
for e in myDict :
    print( "키 =", e)
    print( "값 =", myDict[e])
```

2.6 컬렉션 자료형



- 컬렉션(collection) 자료형
  - 여러 자료를 묶어 한꺼번에 저장하고 처리할 수 있도록 지원
  - 시퀀스형
    - 문자열(str)
    - 리스트(list)
    - 튜플(tuple)
  - 매핑형
    - 딕셔너리(dict)
  - 집합형
    - 집합(set, frosenset)

문자열(str)

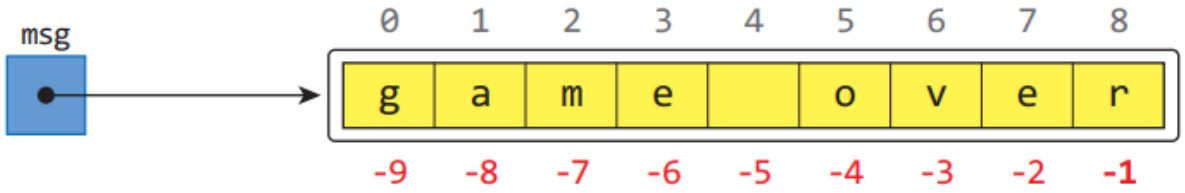
- 문자열

```
msg = 'game over'
hi = "hello world"
sum = "예전엔 " + hi + " 이제는 " + msg
print(sum)
```

```
C:₩WINDOWS₩system32₩cmd.exe
예전엔 hello world 이제는 game over
```

```
print(msg, '의 첫 글자는 ', msg[0])
print(msg, '의 끝 글자는 ', msg[-1])
```

```
C:₩WINDOWS₩system3...
game over 의 첫 글자는 g
game over 의 끝 글자는 r
```




```
hobby = "테니스"
age = 21
score = 4.5
msg1 = " 당신의 학점은 %4.1f입니다" % score
msg2 = " 취미=%s, 나이=%d, 학점=%f" % (hobby, age, score)
```

리스트(list)



- 스마트한 배열

```
big3 = [ ]
lotto = [23, 34, 11, 42, 9]
big4 = [ '블랙핑크', '뉴진스', '에스파', '아이브' ]
print("lotto[1] = ", lotto[1])
big4[2] = '르세라핌'
```

```
big3.append("알라딘")
big3.append("엘사")
big3.append("안나")
```

메소드	설명
s.append(item)	항목 item을 리스트 s의 맨 뒤에 추가한다.
s.extend(lst)	리스트 lst를 s에 추가한다.
s.count(item)	리스트에서 항목 item의 개수를 세고 그 개수를 반환한다.
s.index(item,[시작],[종료])	리스트에서 항목 item을 찾아 가장 작은 인덱스를 반환한다. 탐색의 시작 위치와 종료 위치를 지정할 수도 있다.
s.insert(pos, item)	pos 위치에 항목 item을 삽입한다.
s.pop(pos)	pos 위치의 항목을 s에서 꺼내고 반환한다.
s.remove(item)	항목 item을 s에서 제거한다.
s.reverse()	리스트 항목의 순서를 뒤집는다.
s.sort([key], [reverse])	항목을 정렬한다.

튜플(tuple)



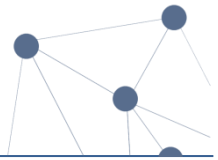
- 리스트(list)와 동일하지만 크기나 값을 변경할 수 없음

```
t = ( 0, 3, 7 )                    # 항목이 3개인 튜플
a = ( 2 )                          # 항목이 1개인 튜플
b = ( 'game', 1, 3.14, 2019 )      # 항목이 4개인 복합 튜플
```

```
print (" 취미=%s, 나이=%d, 학점=%f" % (hobby, age, score))          # 튜플
```

- 메모리는 효율적 (3장)

딕셔너리(dict)



- 키(key)와 관련된 값(value)로 이루어진 항목(entry)들의 집합

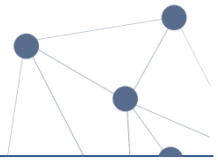
```
map = { '김연아':'피겨', '류현진':'야구', '쿠드롱':'당구', '메시':'축구' }
print(map)
print('쿠드롱이 뭐하는 사람이지? ', map['쿠드롱'])
```

```
C:₩WINDOWS₩system32₩cmd.exe
{'김연아': '피겨', '류현진': '야구', '쿠드롱': '당구', '메시': '축구'}
쿠드롱이 뭐하는 사람이지?  당구
```

```
map['나달'] = '테니스'                          # 맵에 항목 추가
map.update({'최민영':'여자야구', '고진영':'골프'})    # 여러 항목 추가
print(map)
```

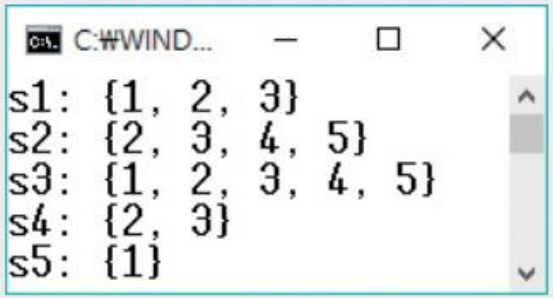
```
C:₩WINDOWS₩system32₩cmd.exe
{'김연아': '피겨', '류현진': '야구', '쿠드롱': '당구', '메시': '축구', '나달': '
테니스', '최민영': '여자야구', '고진영': '골프'}
```

집합(set)



- set과 frozenset(내용을 변경할 수 없는 set)

```
s1 = { 1,2,3 }                  # 집합 객체
s2 = { 2,3,4,5 }                # 집합 객체
s3 = s1.union(s2)               # 합집합
s4 = s1.intersection(s2)        # 교집합
s5 = s1 - s2                    # 차집합
print("s1:", s1)
print("s2:", s2)
print("s3:", s3)
print("s4:", s4)
print("s5:", s5)
```

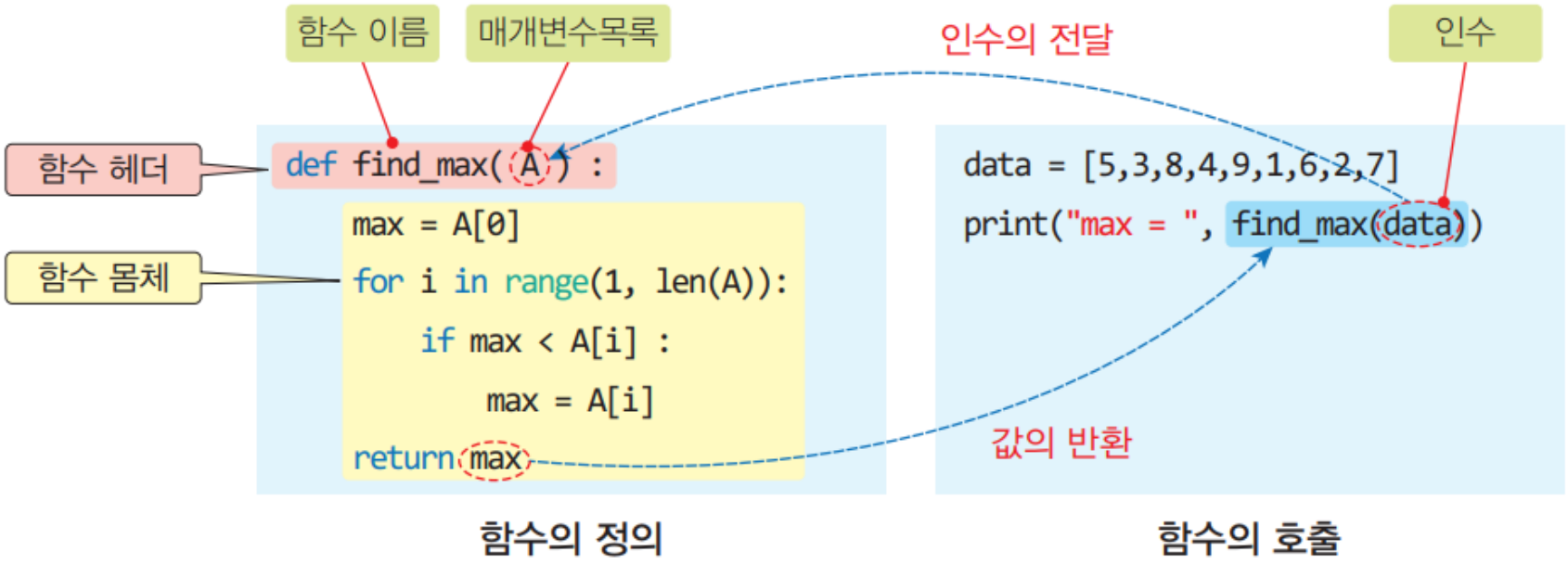



```
s5 = { 3.14 }                   # 원소가 하나인 집합
map = { 3.14 : 'Phi' }          # 엔트리가 하나인 딕셔너리
```

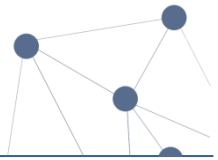
2.7 사용자 정의 함수



- 파이썬 내장 함수: type(), len(), ord() 등
- 사용자 정의 함수



여러 개의 값을 반환할 수 있다



- 최소값과 최대값을 한꺼번에 반환하는 함수

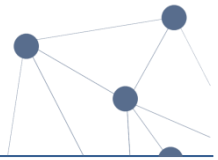
```
01  def find_min_max(A) :                  # 최댓값과 최솟값을 동시에 찾아 반환
02      min = A[0]
03      max = A[0]
04      for i in range(1, len(A)) :        # i : 1 ~ len(A)-1
05          if max < A[i] : max = A[i]     # 최댓값 갱신
06          if min > A[i] : min = A[i]     # 최솟값 갱신
07      return min, max
08  data = [ 5, 3, 8, 4, 9, 1, 6, 2, 7 ]
09  x, y = find_min_max(data)
10  print("(min,max) = ", (x,y))
```

최솟값과 최댓값을 한꺼번에 반환

반환된 값을 순서대로 x와 y에 저장

```
C:₩WINDOWS...
(min,max) =  (1, 9)
```

디폴트 인수와 키워드 인수



- 디폴트 인수 (파라미터)

```
def sum_range(begin, end, step=1) :          # 매개변수 step이 기본 값을 가짐
    sum = 0
    for n in range(begin, end, step) :
        sum += n
    return sum
```

```
print("sum = ", sum_range(1, 10))          # step은 디폴트 값(1)으로 처리됨
```

```
print("sum = ", sum_range(1, 10, 2))       # 정상 호출. step은 2
```

- 키워드 인수

```
print("sum = ", sum_range(step=3, begin=1, end=10))   # 키워드 인수 사용
```

```
print("game ", end=" ")          # 라인피드가 발생하지 않음(키워드 인수 사용)
```

2.8 변수의 범위



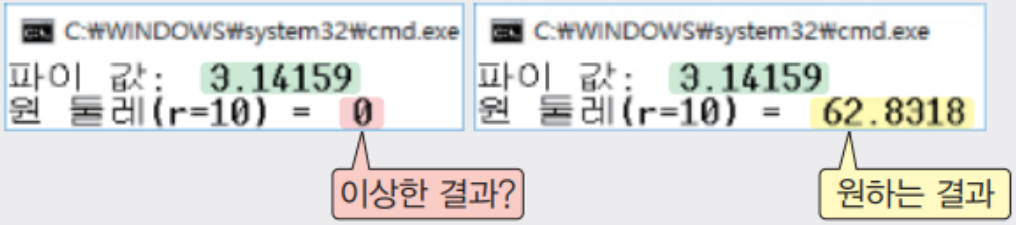
- 내장 범위(built-in scope)
  - 언어의 일부로 정의된 변수와 리터럴들
  - 프로그램의 어디에서나 사용할 수 있다.
- 전역 범위(global scope)
  - 소스 파일의 맨 꼭대기 레벨(함수나 클래스 밖)에서 생성
  - 프로그램의 어디에서나 사용할 수 있다.
- 지역 범위(local scope)
  - 함수나 클래스의 멤버함수(메소드) 안에서 생성
  - 그 안에서만 사용. 함수의 매개변수들도 지역범위
- 인스턴스 범위(instance scope)
  - 클래스의 데이터 멤버로 생성된 변수
  - 클래스 내의 다른 함수들에서 사용할 수 있다.

전역 변수



```
def calc_perimeter(radius) :
    # global perimeter
    print("파이 값: ", pi)                # 파이값 출력 --> OK
    perimeter = 2*pi * radius           # 원 둘레 계산 --> 이상?
```

```
pi = 3.14159                            # 전역변수 pi : 파이값의 근사치
perimeter = 0                           # 전역변수 perimeter : 원 둘레
calc_perimeter(10)
print("원둘레(r=10) = ", perimeter)
```



2.9 모듈과 이름 공간(namespace)

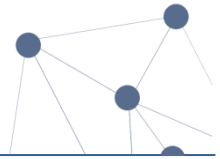


```
# 파일명: min_max.py              # 다음 함수가 min_max.py에 저장되어 있음
def find_min_max(A) :             # 최댓값과 최솟값을 동시에 찾아 반환
    ...                           # 2.7장의 find_min_max()함수 코드와 동일
    return min, max               # 최솟값과 최댓값을 반환
```

```
# 파일명: sum.py                          # 다음 함수가 sum.py에 저장되어 있음
def sum_range(begin, end, step=1) :       # 매개변수 step이 기본 값을 가짐
    ...                                   # 2.7장의 sum_range()함수 코드와 동일
    return sum
```

```
# 파일명: my_job.py
import min_max                    # min_max.py 모듈을 사용함
import sum                        # sum.py 모듈을 사용함

data = [ 5, 3, 8, 4, 9, 1, 6, 2, 7 ]
print("(min,max) = ", min_max.find_min_max(data))    # min_max 모듈의 함수 사용
print("sum = ", sum.sum_range(1, 10))                # sum 모듈의 함수 사용
```



```
from math import pow, sqrt          # math 모듈에서 pow, sqrt 식별자를 사용
result = pow(2,10)                  # pow 식별자를 바로 사용 가능
dist = sqrt(1000)                   # sqrt 식별자를 바로 사용 가능
```

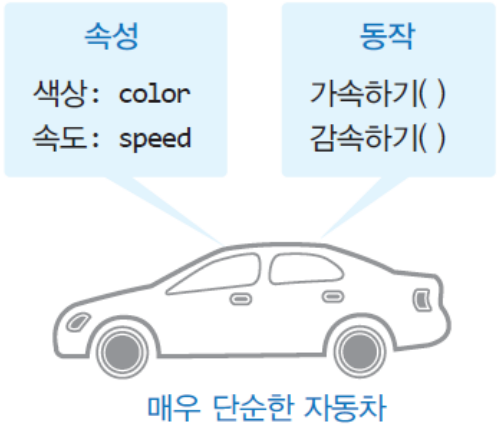
```
# 파일명: my_job.py
from min_max import *          # min_max 모듈의 모든 식별자 사용 가능
from sum import *              # sum 모듈의 모든 식별자 사용 가능

data = [ 5, 3, 8, 4, 9, 1, 6, 2, 7 ]
print("(min,max) = ", find_min_max(data))       # 바로 사용
print("sum = ", sum_range(1, 10))               # 바로 사용
```

2.10 클래스



- 자동차의 추상 자료형과 클래스
  - 속성: 색상과 현재 속도
  - 동작: 가속, 감속



- 자동차의 추상 자료형
  - 데이터
    - 자동차의 색상(color)과 현재 속도(speed)
    - 자동차의 가속이나 감속에 따라 속도가 변경 될 수 있음
  - 연산
    - Car(color, speed): 색상이 color이고 속도가 speed인 새로운 자동차를 만듦
    - speedUp(): 엑셀을 한 번 밟아 속도를 10 증가시킴
    - speedDown(): 브레이크를 한 번 밟아 속도를 10 감소시킴

클래스와 생성자

- 클래스 정의와 생성자

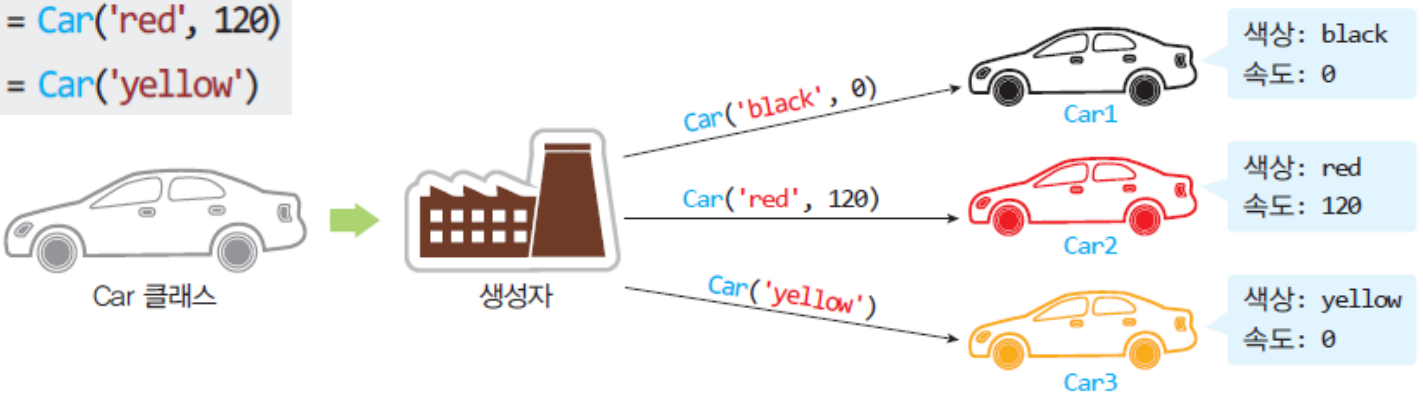
```
01 class Car :
02     def __init__(self, color, speed = 0) :
03         self.color = color
04         self.speed = speed
```

클래스를 만드는 예약어

클래스이름

객체가 만들어질 때마다 자동으로 호출되는 생성자 함수. 멤버 변수들을 정의함

```
car1 = Car('black', 0)
car2 = Car('red', 120)
car3 = Car('yellow')
```



멤버함수 구현과 활용



- 멤버 함수

```
06      def speedUp(self) :
07          self.speed += 10
08
09      def speedDown(self) :
10          self.speed -= 10
```

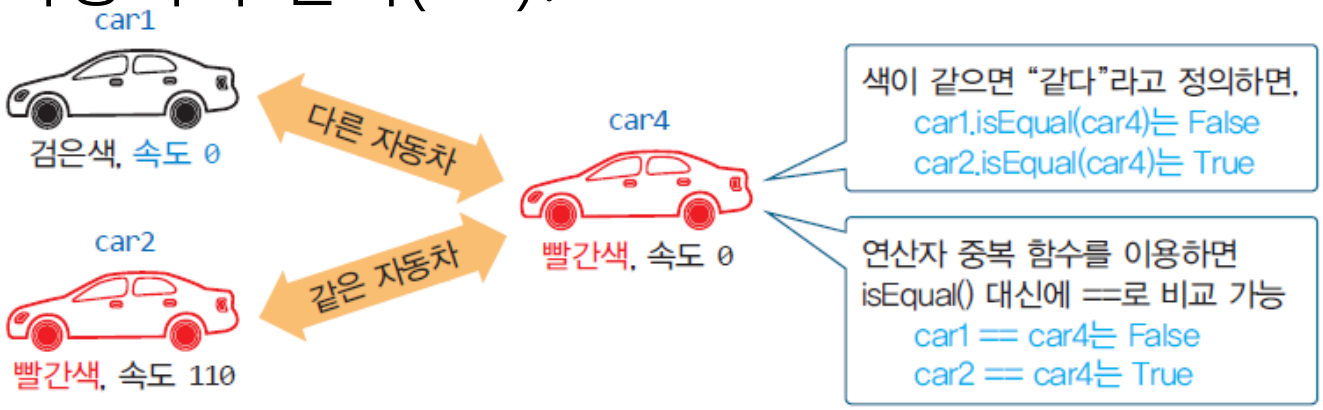
가속 메소드. 이 함수가 호출될 때마다 자동차의 speed 속성이 10 증가함(가속)

감속 메소드. 이 함수가 호출될 때마다 자동차의 speed 속성이 10 감소함(감속)

```
car1.speedUp()      # car1 가속: 속도 10 증가 → car1.speed는 100이 됨
car2.speedDown()    # car2 감속: 속도 10 감소 → car2.speed는 1100이 됨
```

2.11 연산자 중복

- 두 자동차가 같다(==)?



- 일반 멤버 함수 구현

```
def isEqual(self, carB) :
    if self.color == carB.color : return True
    else : return False
```

현재 자동차(self)의 색상과 carB의 색상이 같으면 True, 아니면 False 반환

- 연산자 중복 구현

```
def __eq__(self, carB) :
    return self.color == carB.color
```

== 연산자 중복함수로 구현

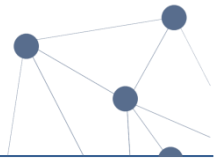
- 문자열로 변환 연산자: __str__

```
def __str__(self) :
    return "color = %s, speed = %d" % (self.color, self.speed)
```

```
print("[car3]", car3)
```

```
C:₩WINDOWS₩system32₩cmd.exe
[car3] color = yellow, speed = 30
```

중복 가능한 연산자들

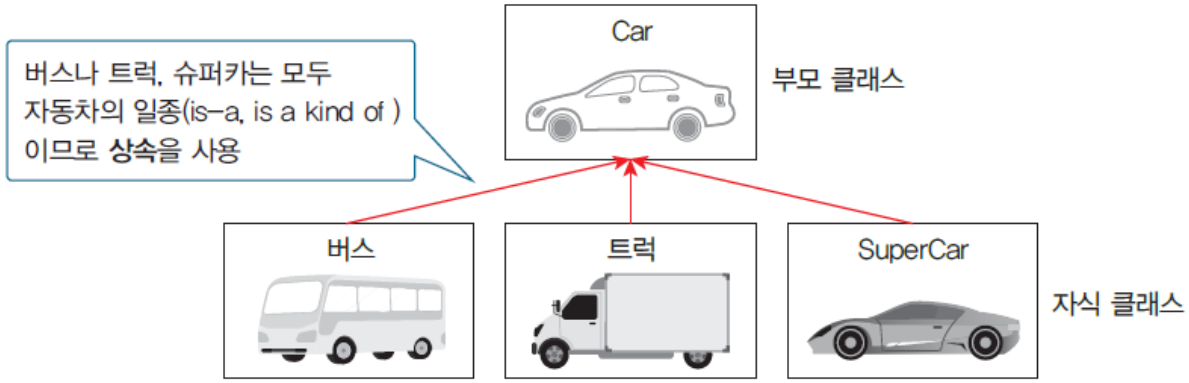


[표 2.1] 파이썬 클래스의 연산자 중복정의 표 단항 연산자 이항 연산자

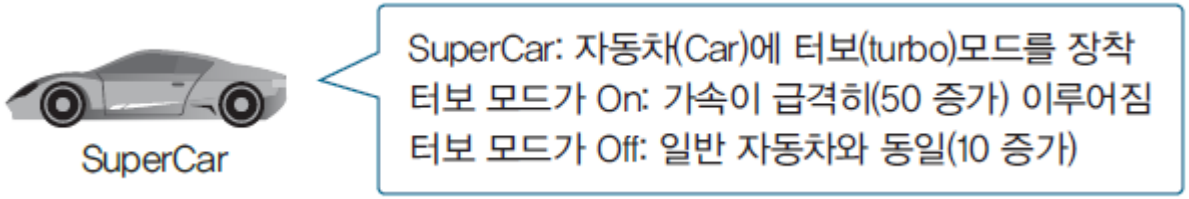
Operation	Class Method	Operation	Class Method
str(obj)	__str__(self)	obj + rhs	__add__(self, rhs)
len(obj)	__len__(self)	obj − rhs	__sub__(self, rhs)
item in obj	__contains__(self, item)	obj * rhs	__mul__(self, rhs)
y = obj[idx]	__getitem__(self, idx)	obj / rhs	__truediv__(self, rhs)
obj[idx] = val	__setitem__(self,idx, val)	obj // rhs	__floordiv__(self, rhs)
		obj % rhs	__mod__(self, rhs)
obj == rhs	__eq__(self, rhs)	obj ** rhs	__pow__(self, rhs)
obj < rhs	__lt__(self, rhs)	obj += rhs	__iadd__(self, rhs)
obj <= rhs	__le__(self, rhs)	obj −= rhs	__isub__(self, rhs)
obj != rhs	__ne__(self, rhs)	obj *= rhs	__imul__(self, rhs)
obj > rhs	__gt__(self, rhs)	obj /= rhs	__itruediv__(self, rhs)
obj >= rhs	__ge__(self, rhs)	obj //= rhs	__ifloordiv__(self, rhs)
		obj %= rhs	__imod__(self, rhs)
		obj **= rhs	__ipow__(self, rhs)

2.12 상속

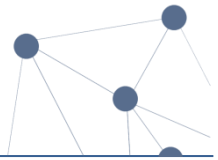
- 자동차 클래스들의 상속 관계



- SuperCar 클래스



클래스의 상속



- 상속의 선언, 생성자, 멤버 추가

```
01  from Car import Car
02  class SuperCar(Car) :
03      def __init__(self, color, speed = 0, bTurbo = True) :
04          super().__init__(color, speed)
05          self.bTurbo = bTurb
06
07      def setTurbo(self, bTurbo = True) :
08          self.bTurbo = bTurbo
```

Car.py 모듈의 Car 포함

Car를 상속한 새로운 클래스 SuperCar 정의

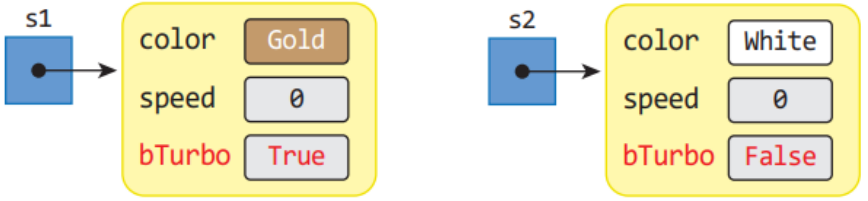
부모의 생성자를 직접 호출

새로운 멤버 추가

생성자

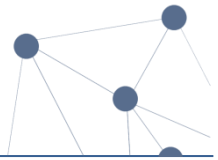
터보 모드(self.bTurbo)를 켜거나 꺼기 위한 새로운 메소드 추가

```
s1 = SuperCar("Gold", 0, True)
s2 = SuperCar("White", 0, False)
s1.speedUp()
s2.speedUp()
print("슈퍼카1:", s1)
print("슈퍼카2:", s2)
```




```
C:₩WINDOWS₩system32₩cmd.exe
슈퍼카1: color = Gold, speed = 10
슈퍼카2: color = White, speed = 10
```

재정의(Overriding)



- 메소드의 재정의

```
07      def setTurbo(self, bTurbo = True) :
08          self.bTurbo = bTurbo
09
10      def speedUp(self) :
11          if self.bTurbo:
12              self.speed += 50
13          else :
14              super().speedUp()
```

터보 모드(self.bTurbo)를 켜거나 꺼기 위한 새로운 메소드 추가

SuperCar의 가속 동작이 Car와 다르므로 부모에 정의된 speedUp() 메소드를 재정의.
터보 모드이면 속도를 50 증가시키고, 아니면 부모(Car)의 speedUp()을 호출

```
s1 = SuperCar("Gold", 0, True)
s2 = SuperCar("White", 0, False)
s1.speedUp()
s2.speedUp()
print("슈퍼카1:", s1)
print("슈퍼카2:", s2)
```

```
C:₩WINDOWS₩system32₩cmd.exe
슈퍼카1: [Gold] [speed = 50] 터보모드
슈퍼카2: [White] [speed = 10] 일반모드
```

파이썬 정리



기능	내용	위치
파일 입출력	• 라인 편집기에서 파일 입출력 기능 • 소스파일의 괄호검사	3.4절 4.3절
슬라이싱 기능	• 스택 내용의 역순출력 • 원형 큐의 내용 출력 함수 display() • 병합정렬의 merge() 함수	4.2절 5.2절 12.4절
클래스 상속	• 원형 큐 클래스를 상속한 원형 덱 클래스의 구현 • 이진탐색트리를 이용한 맵 클래스를 상속하여 AVL 트리를 이용한 맵 클래스를 구현 (멤버 함수 재정의 포함)	5.5절 9.4절
연산자 중복	• 집합 클래스에서 원소의 비교 • 맵의 응용에서 엔트리를 문자열로 변환	7.3절 7.7절
딕셔너리와 집합 사용 예	• 나의 단어장에서 딕셔너리를 사용한 구현 • 딕셔너리와 집합을 이용한 그래프 표현 • 그래프의 탐색, 신장트리, 연결성분 검사 문제에서 그래프 표현 방법으로 사용	7.7절 10.2절 10장
람다 함수	• Kruskal의 최소비용 신장트리 알고리즘에서 간선 리스트를 가중치의 내림차순으로 정렬	11.3절
모듈 사용	• time 모듈: 알고리즘 시간 측정 • queue 모듈: 큐의 응용 기수정렬 • heapq 모듈: 힙 트리 • collections 모듈: 그래프 너비우선탐색에서 deque 클래스를 큐로 사용 • random 모듈: 난수 발생(random.randint()) • sys 모듈: 정수 무한대(sys.maxsize) 사용 • copy 모듈: 깊은 복사(copy.deepcopy) 사용	1.3절 5.3절 12.7절 8.5절 10.3절 12.7절 11.3절 11.4절

Summary



자료형 (컬렉션)

- (시퀀스) str, list, tuple
- (매핑) dict
- (집합) set

제어 / 반복

- If / elif / else
- for / while (continue/break)

함수 + 변수

- 파라미터 받아서(없을 수도) -> 연산 -> return(없을 수도)
- 지역 변수(함수 내에서만 사용) / 전역 변수(코드 전체에서 사용. "global"로 함수에서 조작 가능

모듈 / 클래스+상속

- 모듈의 함수/변수를 import 해와서 사용 가능
- 클래스를 import 해와서 속성값을 설정하고 함수 활용 가능
- 클래스 상속을 통한 재사용성 / 시스템 통일성 유지 가능



감사합니다!